

CSE221: Algorithms

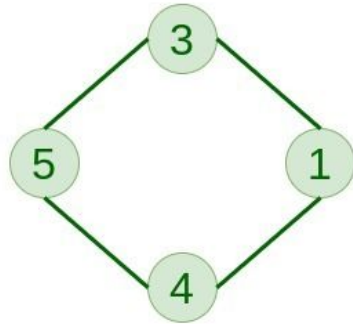
Strongly Connected Components

Prepared by: A B M Muntasir Rahman [MUNR]

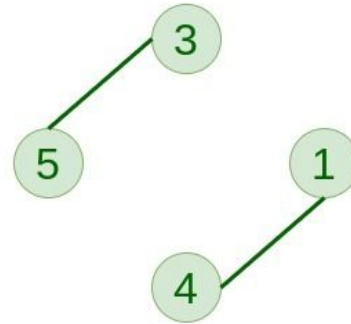


Inspiring Excellence

Connected Components



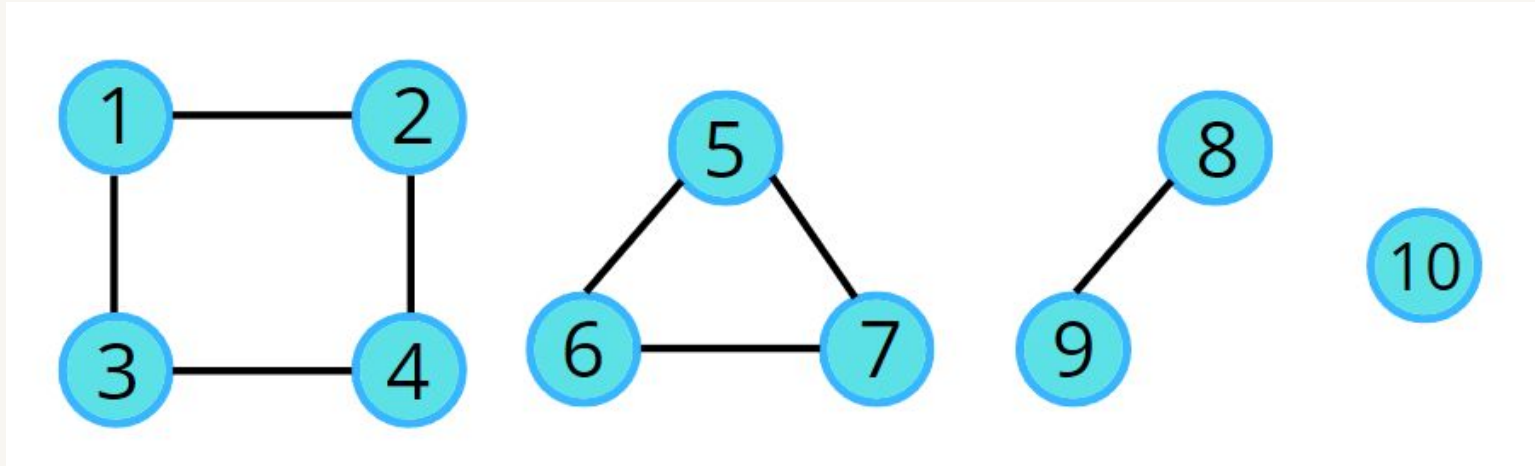
Connected Graph



Disconnected Graph

GG

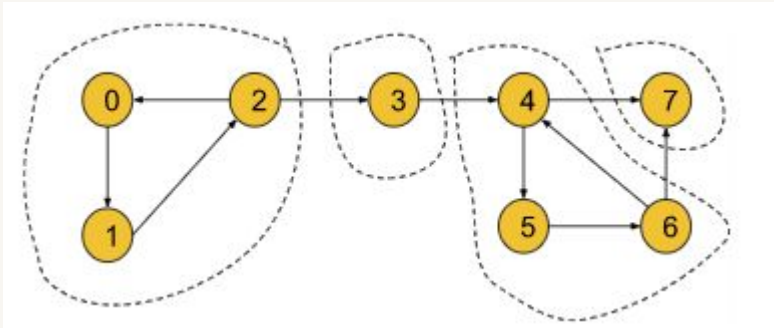
Connected Components



How many connected components here? How do we visit all of them?

Strongly Connected Components

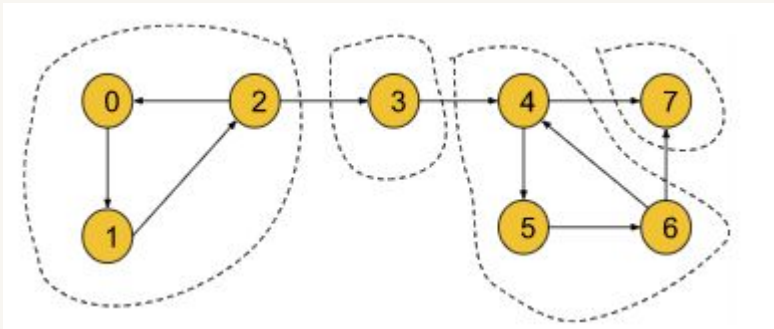
- A component is called a Strongly Connected Component (SCC) only if for every possible pair of vertices (u, v) inside that component, u is reachable from v and v is reachable from u .



In this graph, $[0, 1, 2]$ is a strongly connected component. Here, we can reach from 0 to 1 and vice versa, 1 to 2 and vice versa and 2 to 0 and vice versa.

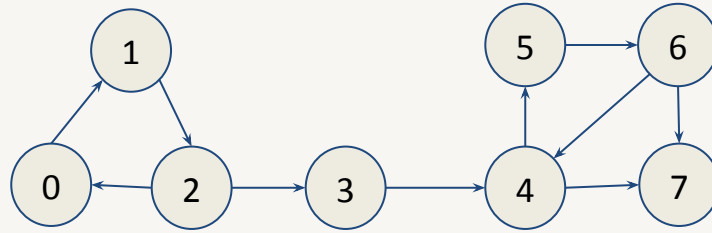
Strongly Connected Components

- Similarly, in this graph, there are 4 strongly connected components, $[0, 1, 2]$, $[3]$, $[4, 5, 6]$ and 7 .



Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Start Time

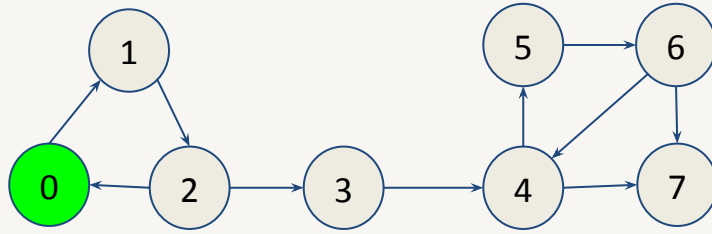
0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Start Time

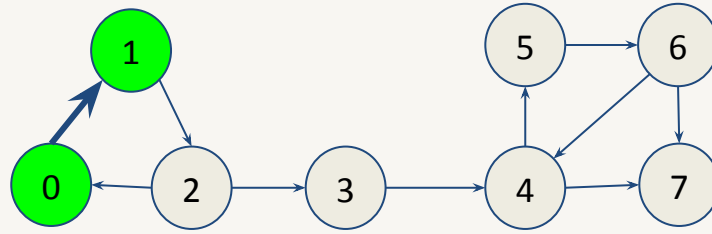
1	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Start Time

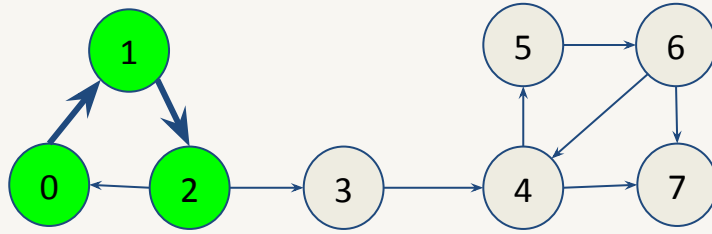
1	2	0	0	0	0	0	0
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	0	0	0	0	0
0	1	2	3	4	5	6	7

Start Time

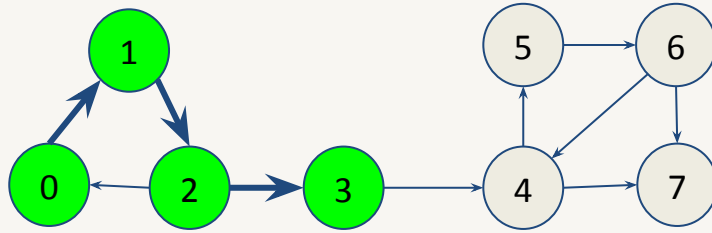
1	2	3	0	0	0	0	0
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	0	0	0	0
0	1	2	3	4	5	6	7

Start Time

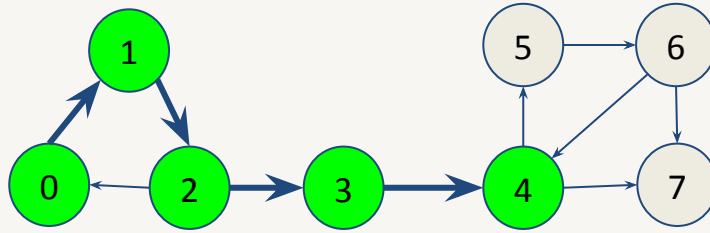
1	2	3	4	0	0	0	0
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	0	0	0
0	1	2	3	4	5	6	7

Start Time

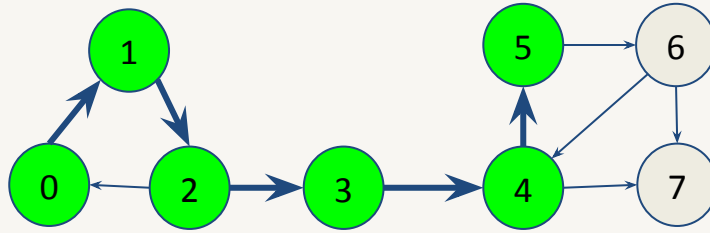
1	2	3	4	5	0	0	0
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	1	0	0
0	1	2	3	4	5	6	7

Start Time

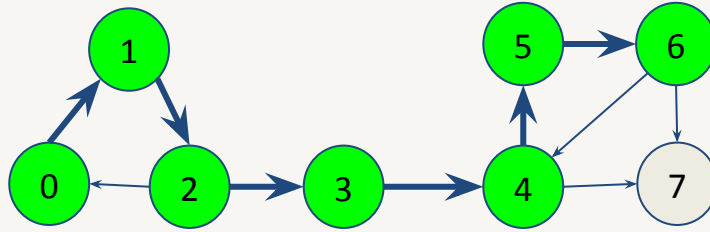
1	2	3	4	5	6	0	0
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	1	1	0
0	1	2	3	4	5	6	7

Start Time

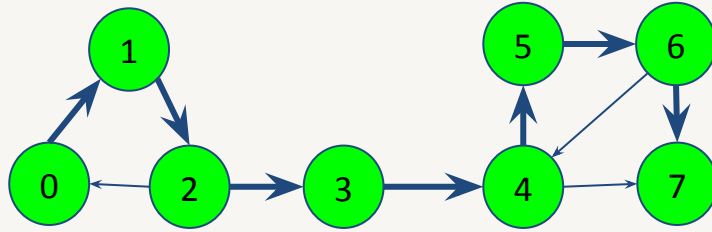
1	2	3	4	5	6	7	0
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Start Time

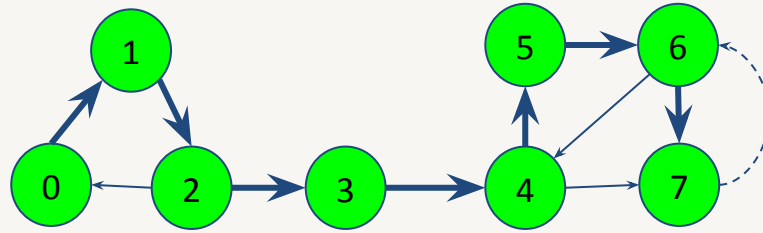
1	2	3	4	5	6	7	8
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Start Time

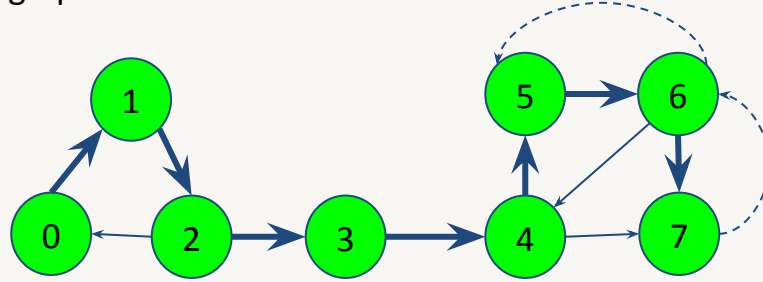
1	2	3	4	5	6	7	8
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	0	0	9
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Start Time

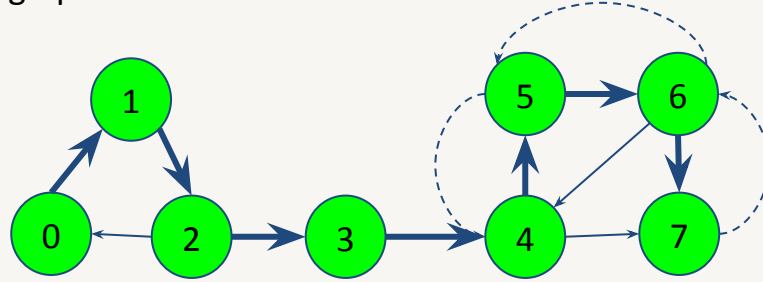
1	2	3	4	5	6	7	8
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	0	10	9
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Start Time

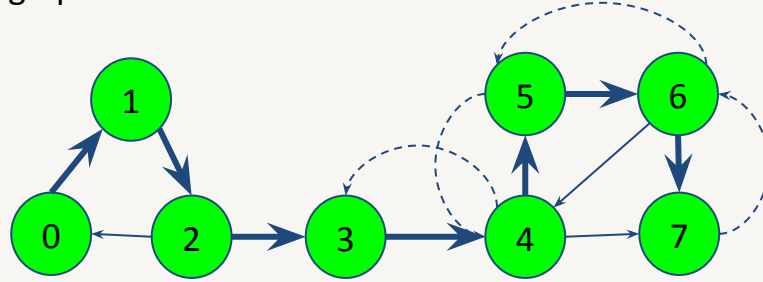
1	2	3	4	5	6	7	8
0	1	2	3	4	5	6	7

End Time

0	0	0	0	0	11	10	9
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Start Time

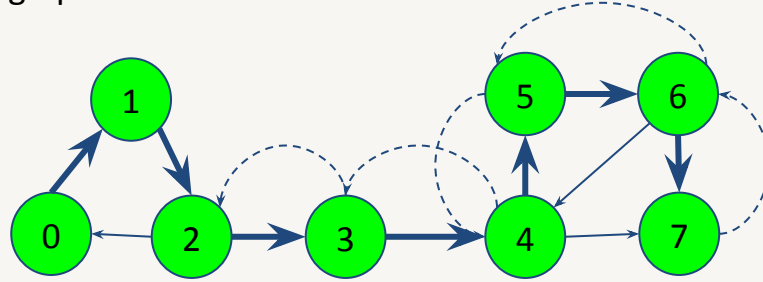
1	2	3	4	5	6	7	8
0	1	2	3	4	5	6	7

End Time

0	0	0	0	12	11	10	9
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Start Time

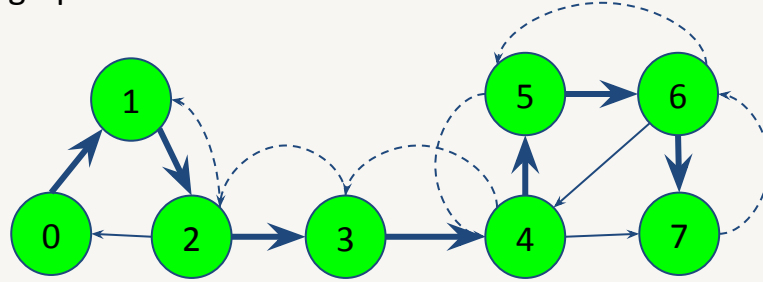
1	2	3	4	5	6	7	8
0	1	2	3	4	5	6	7

End Time

0	0	0	13	12	11	10	9
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Start Time

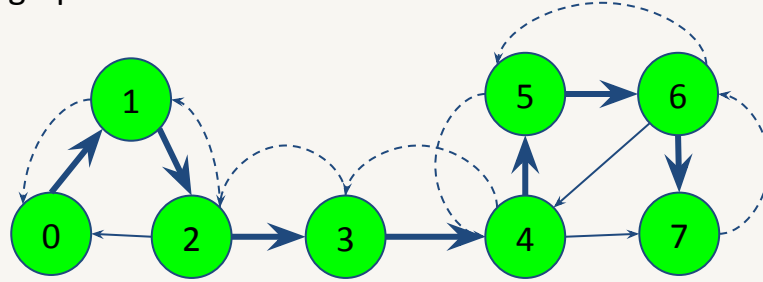
1	2	3	4	5	6	7	8
0	1	2	3	4	5	6	7

End Time

0	0	14	13	12	11	10	9
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Start Time

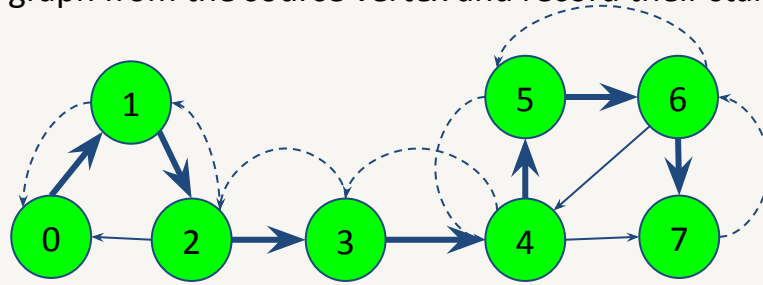
1	2	3	4	5	6	7	8
0	1	2	3	4	5	6	7

End Time

0	15	14	13	12	11	10	9
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Run DFS on the initial graph from the source vertex and record their start and end times.



Visited

1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Start Time

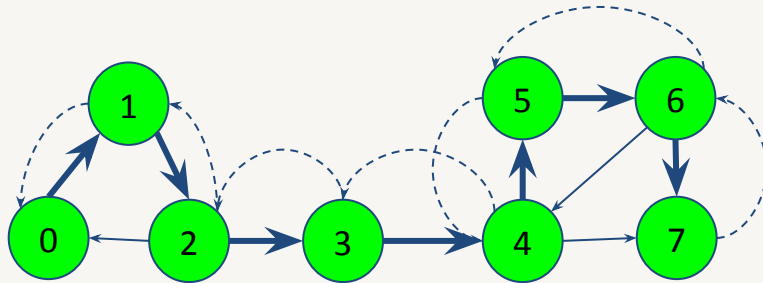
1	2	3	4	5	6	7	8
0	1	2	3	4	5	6	7

End Time

16	15	14	13	12	11	10	9
0	1	2	3	4	5	6	7

Kosaraju's Algorithm

- Based on descending order of end time of vertices, put them in a reference stack.



So, descending order of vertices according to end time is 0, 1, 2, 3, 4, 5, 6, 7

End Time

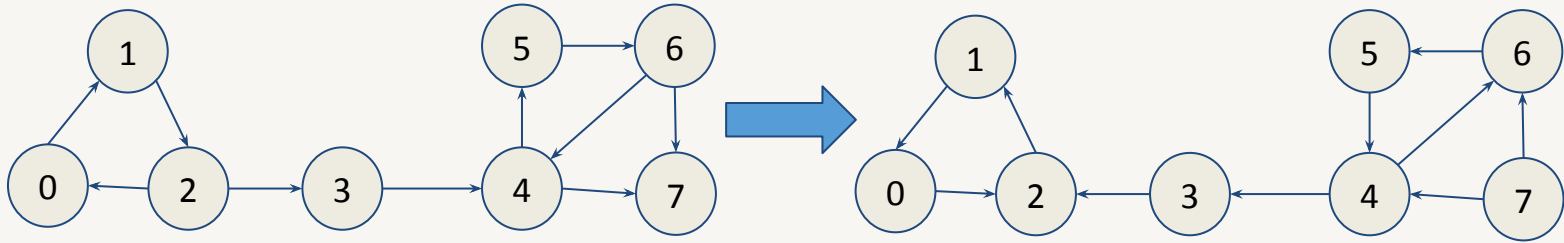
16	15	14	13	12	11	10	9
0	1	2	3	4	5	6	7

7 6 5 4 3 2 1 0

Reference Stack

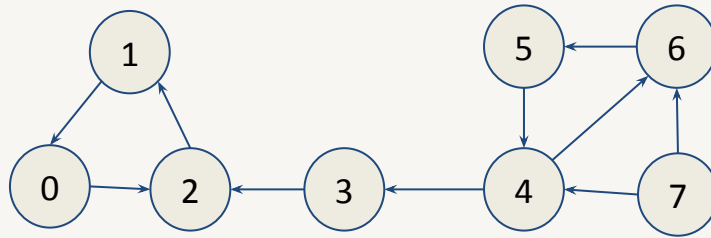
Kosaraju's Algorithm

- Now, transpose the given graph i.e. edge (u, v) becomes (v, u) .



Kosaraju's Algorithm

- Run DFS according to the order of vertices from the reference stack on transposed graph.



Visited

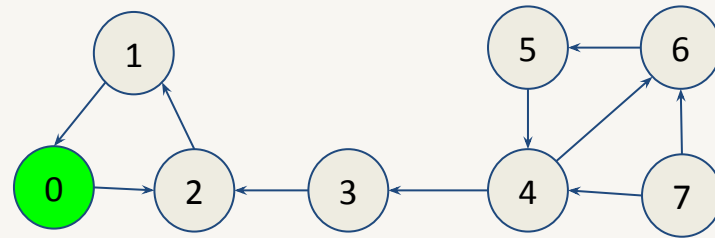
0	0	0	0	0	0	0	1
0	1	2	3	4	5	6	7

7	6	5	4	3	2	1	0
---	---	---	---	---	---	---	---

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 0 in this case and run DFS starting from 0 on the transposed graph.



DFS(0)

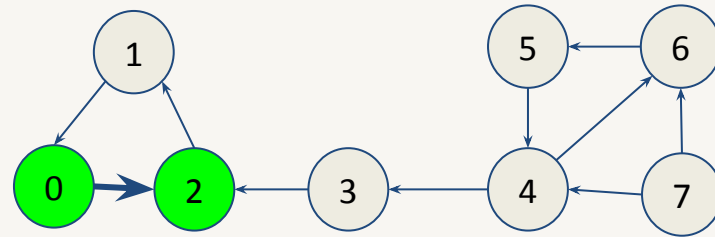
Visited

1	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7

7	6	5	4	3	2	1
---	---	---	---	---	---	---

Reference Stack

Kosaraju's Algorithm



DFS(0) -> DFS(2)

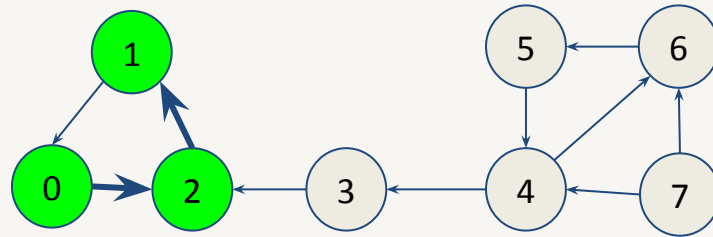
Visited

1	0	1	0	0	0	0	0
0	1	2	3	4	5	6	7

7	6	5	4	3	2	1
---	---	---	---	---	---	---

Reference Stack

Kosaraju's Algorithm



DFS(0) -> DFS(2) -> DFS(1)

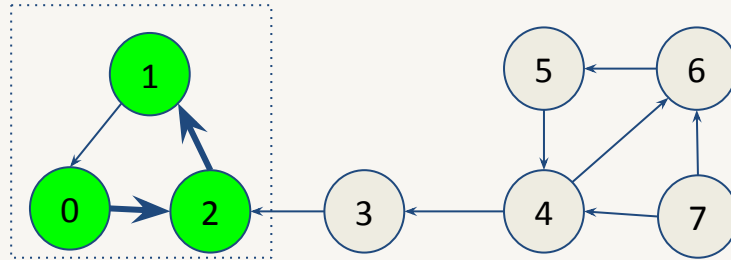
Visited

1	1	1	0	0	0	0	0
0	1	2	3	4	5	6	7

7	6	5	4	3	2	1
---	---	---	---	---	---	---

Reference Stack

Kosaraju's Algorithm



Visited

1	1	1	0	0	0	0	0
0	1	2	3	4	5	6	7

7	6	5	4	3	2	1
---	---	---	---	---	---	---

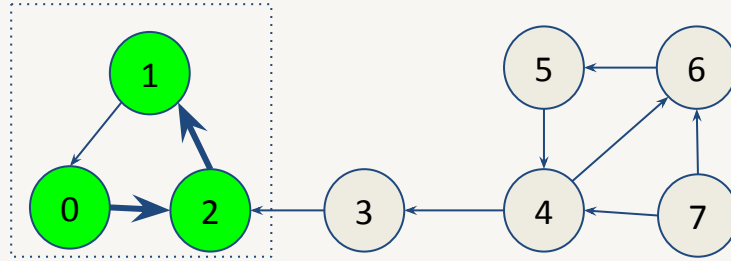
Reference Stack

DFS(0) -> DFS(2) -> DFS(1)

Now, from 1 we can go to 0 however it is already visited. Since there are no unvisited vertex, we'll backtrack to 2 and from 2, we'll backtrack to 0. Therefore, [0, 2, 1] is our first SCC.

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 1 in this case. However, 1 is already visited.



Visited

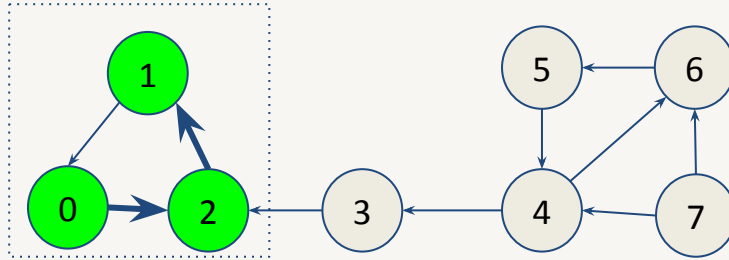
1	1	1	0	0	0	0	0
0	1	2	3	4	5	6	7

7	6	5	4	3	2
---	---	---	---	---	---

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 2 in this case. However, 2 is already visited.



Visited

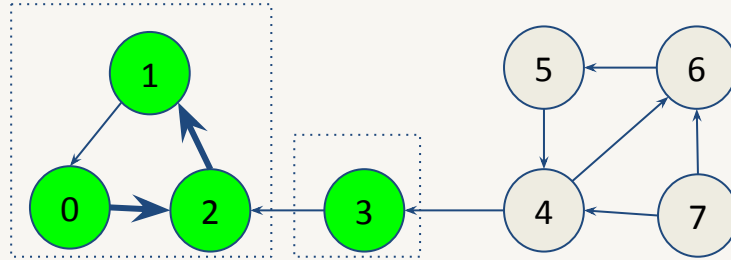
1	1	1	0	0	0	0	0
0	1	2	3	4	5	6	7

7	6	5	4	3
---	---	---	---	---

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 3 in this case. Run DFS from 3.



Visited

1	1	1	1	0	0	0	0
0	1	2	3	4	5	6	7

7	6	5	4
---	---	---	---

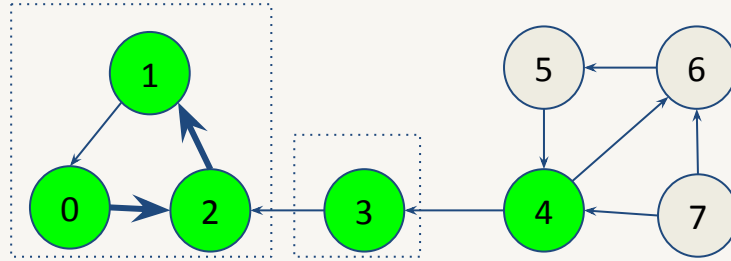
Reference Stack

DFS(3)

From 3, we can go to 2. However, 2 is already visited. As we don't have any other adjacent vertices, 3 is our second SCC.

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 4 in this case. Run DFS from 4.



DFS(4)

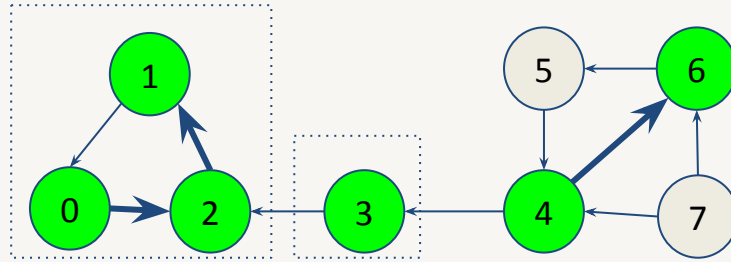
Visited

1	1	1	1	1	0	0	0
0	1	2	3	4	5	6	7

7	6	5
---	---	---

Reference Stack

Kosaraju's Algorithm



DFS(4) -> DFS(6)

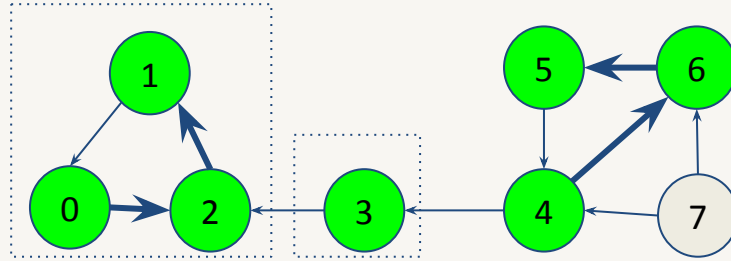
Visited

1	1	1	1	1	0	1	0
0	1	2	3	4	5	6	7

7	6	5
---	---	---

Reference Stack

Kosaraju's Algorithm



Visited

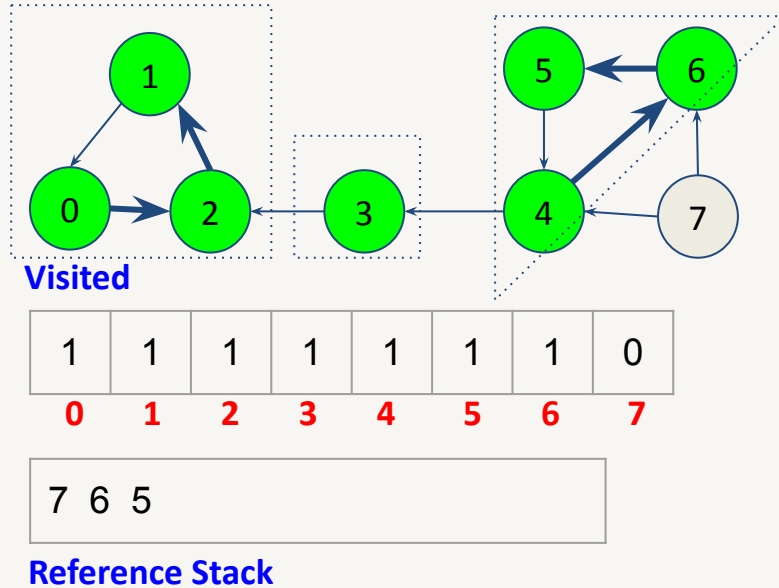
1	1	1	1	1	1	1	0
0	1	2	3	4	5	6	7

7	6	5
---	---	---

Reference Stack

DFS(4) -> DFS(6) -> DFS(5)

Kosaraju's Algorithm

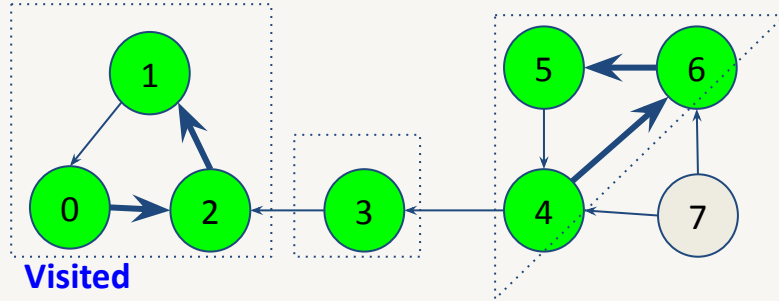


DFS(4) -> DFS(6) -> DFS(5)

From 5 we can go to 4. However, it is already visited. So, we'll backtrack from 5 to 6, and then 6 to 4. Since 4 doesn't have any adjacent vertices, therefore, [4, 6, 5] is our third SCC.

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 5 in this case. However, 5 is already visited.



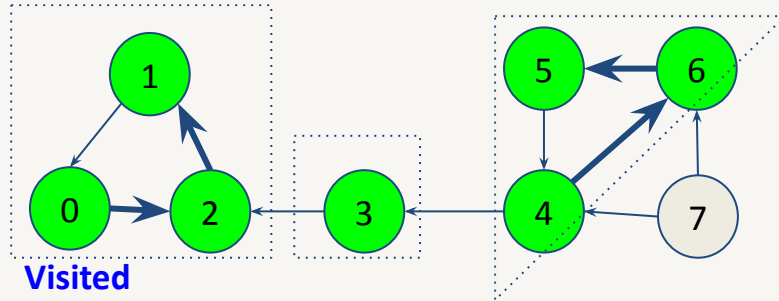
1	1	1	1	1	1	1	0
0	1	2	3	4	5	6	7

7	6
---	---

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 6 in this case. However, 6 is already visited.



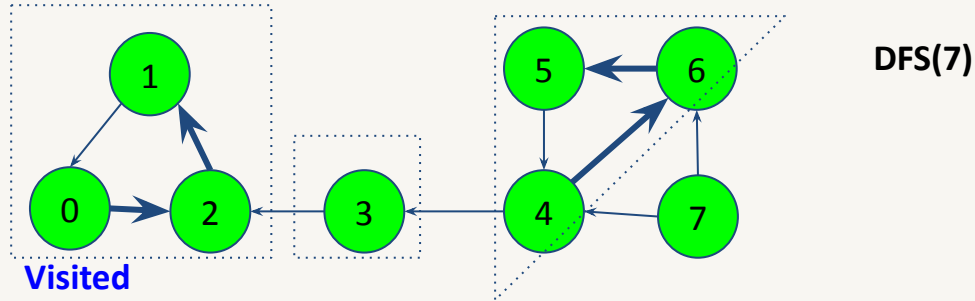
1	1	1	1	1	1	1	0
0	1	2	3	4	5	6	7

7

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 7 in this case. Run DFS from 7.

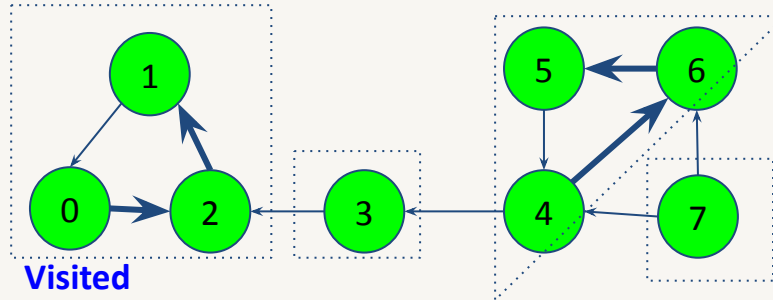


1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7

Reference Stack

Kosaraju's Algorithm

- Pop the vertex from the stack i.e. 7 in this case. Run DFS from 7.



1	1	1	1	1	1	1	1
0	1	2	3	4	5	6	7



Reference Stack

DFS(7)

7 has two adjacent vertices 4 and 6, however, both are already visited. Therefore, 7 is our fourth SCC.

[0, 2, 1]: SCC1

[3]: SCC2

[4, 6, 5]: SCC3

[7]: SCC4

This is Kosaraju's Algorithm!